

RWA T-Bill Tokenization Protocol

Full-Stack Decentralized Protocol - Blockchain Technologies 2 Final Project

Option C

RWA tokenization platform

Foundry

Anvil-first, no Hardhat

Base Sepolia

L2 deployment target

Problem and Product

Tokenize a simplified T-Bill reserve product as **TBILL**

Keep minting issuer-gated and reserve-evidence driven

Give users a vault, AMM liquidity, governance, and indexed activity

Build a protocol another engineer can test, audit, deploy, and extend

Requirement Coverage

Course requirement	Protocol implementation
Upgradeability	UUPS <code>TBillToken</code> , V1 to V2 upgrade test
Token standards	ERC20Votes, ERC20Permit, ERC721, ERC4626
DeFi primitive	From-scratch $x*y=k$ AMM with 0.3% fee
Oracle	Chainlink-compatible adapter with staleness reverts
Governance	Governor + 2 day Timelock lifecycle
Indexing and UI	The Graph plus Vite React dApp

Architecture at a Glance

Frontend

RainbowKit, Wagmi, Viem,
network checks

Core Protocol

TBILL, vault, AMM, oracle,
treasury

Governance

Votes token, Governor,
Timelock

Data Layer

Subgraph entities and
GraphQL queries

Deployment

Foundry scripts for Anvil
and Base Sepolia

Smart Contracts

TBillToken : upgradeable RWA ERC20 with issuer minting

RwaStableAMM : $x*y=k$ liquidity and swaps

RwaGovernor and **TimelockController** : DAO-controlled admin layer

TBillVault : ERC4626 vault for tokenized yield

OracleAdapter : price and reserve feeds with freshness checks

ProtocolFactory : CREATE and CREATE2 asset deployment

Upgradeability Design

Only the RWA asset token is upgradeable to keep upgrade risk narrow

V2 appends `complianceUri` without reordering inherited storage

Foundry test proves V1 state survives `upgradeToAndCall`

V1 stores core ERC20, roles, pause state, registry, and reserved gap

Upgrade authority is `UPGRADER_ROLE`, handed to Timelock after deploy

RWA Minting and Reserve Evidence

Reserve docs

Off-chain custody
evidence exists

Certificate

Issuer mints ERC721
metadata URI

Oracle read

Adapter checks feed
freshness

Mint TBILL

``ISSUER_ROLE`` required

Index event

Subgraph records transfer
and certificate

ERC4626 Vault

Users deposit `TBILL` and receive ERC4626 vault shares

Deposit, mint, withdraw, redeem, and yield flows are `nonReentrant`

Vault actions are indexed by the subgraph

Yield is reported by a role-gated manager through real token transfer

Rounding behavior is covered with fuzz tests

AMM Built from Scratch

Constant product invariant: $\text{reserve0} * \text{reserve1} = k$

LP token accounting for add and remove liquidity

Deadline protection for stale transactions

Swap fee: 0.3%, retained in the pool for LPs

Slippage protection with amountOutMin

Invariant tests verify k never decreases after swaps

Oracle and Governance

Oracle adapter rejects stale, zero, negative, and incomplete feed data

Timestamp clock gives human-readable voting windows

Voting period: 1 week

Governance token uses ERC20Votes + ERC20Permit

Voting delay: 1 day

Quorum: 4%, proposal threshold: 1%, Timelock delay: 2 days

Security Model

Control	Implementation
Privileged functions	OpenZeppelin AccessControl or Governor/Timelock
Reentrancy	ReentrancyGuard and CEI on value-moving flows
ERC20 transfers	SafeERC20 in protocol contracts
Forbidden primitives	No tx.origin, no ETH transfer/send, no timestamp randomness
Vulnerability studies	Reentrancy and access-control before/after tests

Testing Evidence

88

Total Foundry tests pass

92.31%

Line coverage across production contracts

5+

Invariant categories including AMM, vault, treasury, roles

Unit tests cover public/external functions and revert paths

Fuzz tests cover swaps, ERC4626 rounding, and voting power

Fork hooks cover USDC, Uniswap V2 router, and Chainlink feed integrations

Frontend dApp

Wallet connection through RainbowKit with MetaMask and WalletConnect support

Reads balances, voting power, delegate address, vault state, AMM reserves, oracle data

Formats wallet rejection, insufficient balance, and RPC failures into readable messages

Wrong-network detection and switch prompt

Writes deposit, withdraw, add liquidity, swap, delegate, vote, and issuer actions

Subgraph and Data Model

Indexes asset transfers, vault deposits and withdrawals, swaps, liquidity positions, proposals, votes, and treasury payments

Provides documented GraphQL queries for demo and grading review

Frontend activity section reads indexed data from The Graph rather than only contract state

Contract state remains source of truth; subgraph is an indexed read layer

Deployment and DevOps

Foundry scripts: `Deploy.s.sol`,
`UpgradeTBillToken.s.sol`,
`VerifyDeployment.s.sol`, `GasCompare.s.sol`

CI runs compile, tests, coverage, formatting, Solhint, Slither, frontend build, and no-Hardhat guard

Anvil deploy and verification already run locally

Deployment manifests written to
`deployments/<chainId>.json`

Base Sepolia deploy requires local secrets and block explorer API key

Gas and Optimization

Operation	Local gas	Optimization note
TBill mint	26,003	Role-gated ERC20 mint path
Vault deposit	79,670	ERC4626 share accounting plus SafeERC20
AMM addLiquidity	152,599	LP mint, reserve sync, transfer checks
AMM swapExactIn	17,308	Constant-product math and fee path
Oracle latestPrice	2,736	Read-only adapter validation

Demo Flow

Connect

Wallet connects to Anvil or Base Sepolia

Inspect

Balances, votes, vault shares, reserves

Deposit/Swap

State-changing protocol transaction

Govern

Delegate and cast a vote

Index

Review activity through subgraph view

Team Ownership

Team member	Primary ownership	Defense responsibility
Tokhtamishov Bekhruz	Contracts, security, tests, deployment scripts	Contract architecture, vulnerabilities, governance, testing
Zhangir Muslimov	Frontend, subgraph, documentation, presentation, gas report	UI flows, indexing, diagrams, deployment docs, user demo

Both team members must understand the whole system at architecture level for Q&A.

Defense Talking Points

Why UUPS is limited to `TBillToken` instead of every contract

How ERC20Votes checkpoints defend against flash-loan voting

How AMM and vault invariants are tested

How Timelock removes direct deployer admin control

Why testnet reserve feeds are mocks and what production replacement requires

How frontend state differs from subgraph indexed activity

Final Status

Implemented

Full smart contract, frontend, subgraph, docs, CI, and scripts

Verified

Anvil deploy, verification script, build, test, coverage, subgraph build

Remaining

Base Sepolia secrets, final deploy, explorer links, final Slither appendix